COMPARISON OVERVIEW // 概要

クリーンアーキテクチャと ヘキサゴナルアーキテクチャの違い

「同じような図に見えるが何が違うのか？」という疑問を解消します。
結論から言うと、**クリーンはヘキサゴナルの思想を包含し、より厳密な規約を規定した後発の体系**です。

ヘキサゴナル (Ports & Adapters)

2005年 / Alistair Cockburn

- **構造:** 「内側（コア）」と「外側（外部技術）」の2領域
- **思想:** 外部（Web/DB）を取り外し可能なプラグインにする
- **特徴:** シンプルで現場適用しやすい（Port = I/F, Adapter = 実装）
- **UIの扱い:** ControllerがUseCaseの戻り値を直接受け取って返却

クリーンアーキテクチャ

2012年 / Robert C. Martin

- **構造:** 厳密な同心円の4階層（Entities / UseCases / Adapters / Drivers）
- **思想:** ヘキサゴナルやオニオン等を統合し、包括的なルールを策定
- **特徴:** 規約が詳細（EntitiesとUseCasesの分離、Interactor）
- **UIの扱い:** Presenterパターンによる出力（UI整形）の完全分離

構造比較

メンタルモデルの違い:「内外の対称性」vs「厳密な4層同心円」

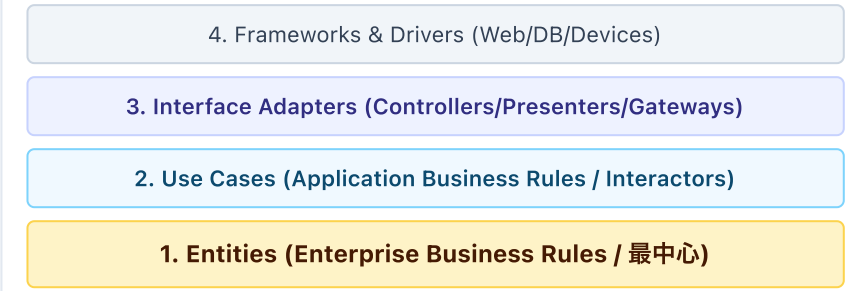
どちらも「外から内への依存」ですが、コア内部のモデル化に大きな差があります。

【ヘキサゴナル】多角形モデル（内と外）



焦点: 外部技術（HTTP/SQL）を「境界のポート」で遮断すること。コア内部の層分けはチーム裁量。

【クリーン】同心円4層モデル（詳細な階層）



焦点:「企業全体のルール（Entities）」と「アプリ固有のフロー（Use Cases）」を厳密に分離すること。

用語対比

構成要素・用語の完全マッピング

名前が異なるだけで、同じ目的を持つパーツと、クリーン特有のパーツを整理します。

関心事	ヘキサゴナルでの名称	クリーンでの名称	役割の違い・補足
ビジネスルール（核）	Domain Model	Entities	ほぼ同義。純粋なドメインオブジェクト。
ユースケース実行役	UseCase Service	UseCase Interactor	ほぼ同義。業務フローを統括する進行役。
入力の受付規約	Inbound Port	Input Port / Boundary	同義。Controllerが呼び出すI/F。
出力・永続化の規約	Outbound Port	Output Port / Gateway I/F	クリーンではUI出力用とDB用が別ポートになる。
永続化の実装	Driven Adapter (DB)	Gateway (Repository Impl)	同義。SQLを発行してDBアクセスする部品。
画面表示データの整形	Controller内で実施	**Presenter / ViewModel**	**★ クリーン独自の分離要素。**

最大の違い: 出力（UI・表示）のフロー設計

ヘキサゴナルは「同期的な戻り値」を基本としますが、クリーンは「Presenterへのディスパッチ」を提唱します。

ヘキサゴナルの出力フロー (Request-Response型)

```
// Controllerが戻り値を受け取って画面に渡す
public class OrderController {
    public View createOrder(Request req) {
        OrderResult result = useCase.execute(cmd);
        // Controller自身がレスポンスやViewを決定
        return new View("orderSuccess.jsp", result);
    }
}
```

特徴: 直感的でSpring MVCなどのWebフレームワークと極めて相性が良い。

クリーンの出力フロー (Presenter分離型)

```
// Interactorが出力ポート（Presenter）へ通知
public class OrderInteractor implements OrderInputPort {
    private final OrderOutputPort presenter;

    public void execute(OrderRequestModel req) {
        OrderResponseModel res = ...;
        // 戻り値を返さず、OutputPortへ渡す
        presenter.present(res);
    }
}
```

特徴: Interactorは画面描画やHTTPレスポンスの都合を1ミリも知らず、PresenterがViewModelへ変換する。

Javaによるユースケース実装コード比較

構造のシンプルさ（ヘキサゴナル）と、関心の厳密な分離（クリーン）の違いです。

【ヘキサゴナル】UseCaseService

```
public class RegistShainService
    implements RegistShainUseCase {

    private final ShainRepositoryPort repo;

    @Override
    public ShainId execute(RegistShainCommand cmd) {
        Shain shain = new Shain(cmd.getName());
        repo.save(shain);
        return shain.getId(); // 戻り値を直接返す
    }
}
/* メリット:
   クラス数が最小限でシンプル。可読性が高い。 */
```

【クリーン】Interactor + Presenter

```
public class RegistShainInteractor
    implements RegistShainInputPort {

    private final ShainGateway gateway;
    private final RegistShainOutputPort presenter;

    @Override
    public void execute(RegistShainRequestModel req) {
        Shain shain = new Shain(req.name());
        gateway.save(shain);
        // 戻り値は返さず、Presenterに渡す
        presenter.presentSuccess(shain.getId());
    }
}
/* メリット:
   UI形式（JSON/HTML/CLI）を完全分離可能。 */
```

選定基準

どちらを採用すべきか？ 意思決定の指針

システムの規模とチームの習熟度に応じた現実的な使い分け基準です。

🎯 ヘキサゴナルが最適なケース

- **Web API / マイクロサービス中心のシステム**
- MVCから初めてアーキテクチャ移行するチーム
- ファイル数やクラス数を増やしすぎず、学習コストを抑えたい
- Spring BootなどのDIコンテナと自然に統合したい

🎯 クリーンが最適なケース

- **同一ロジックに対して複数のUI（Web, App, CLI）が存在する**
- 業務ルールが極めて複雑で、EntitiesとUseCasesを厳密に分けたい
- 大規模開発で、UIチームとバックエンドチームで完全分業したい
- PresenterとViewModelまで含めた徹底的なUIテストを行いたい

💡 **まとめ:** 本質（「ビジネスロジックを中心に置き、DBや外部をポートでプラグイン化する」）は全く同一です。 **「まずはヘキサゴナルで始め、UIの関心分離や複雑化が進んだ段階でクリーンアーキテクチャのPresenter等を導入する」** のが最も失敗の少ないアプローチです。